

Taller CampusTasks — entorno, actualización y eliminación de tareas

1. Datos

Asignatura	Desarrollo de Software II — Universidad del Valle
Fecha	6 de octubre de 2026
Integrante 1	Juan Felipe Aristizabal — 2459364-3743
Integrante 2	Juan José Bolaños — 2380616
Repositorio (fork)	https://github.com/JuanFelipe-A/campus-task
Rama de entrega	jfa

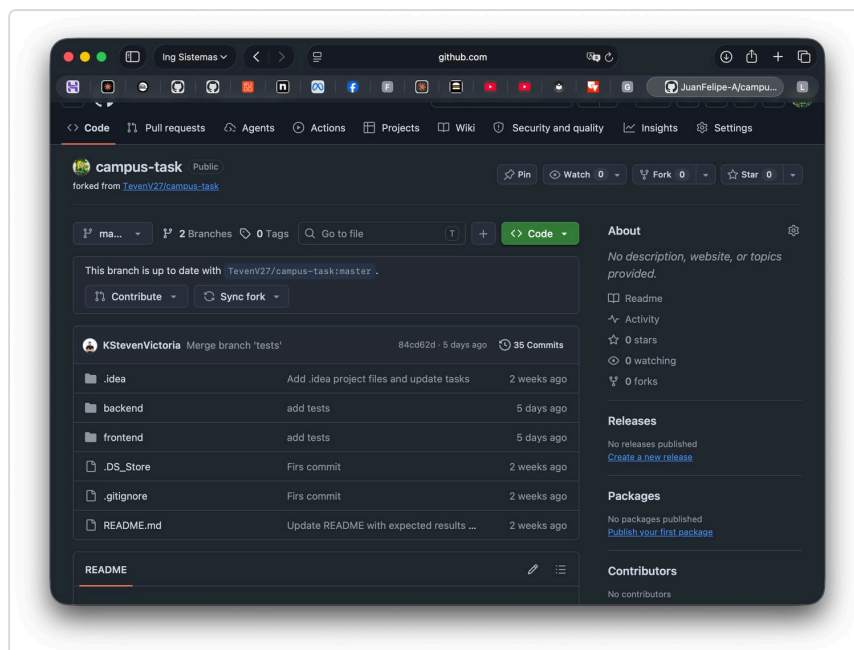
Reparto del trabajo

- Juan Felipe Aristizabal: puesta en marcha del entorno y backend (actualizar y eliminar tareas, con sus pruebas).
- Juan José Bolaños: frontend (actualizar y eliminar tareas desde la interfaz, con sus pruebas).
- Prueba manual de extremo a extremo: realizada en conjunto, sobre el equipo de Juan Felipe (backend, base de datos y frontend).

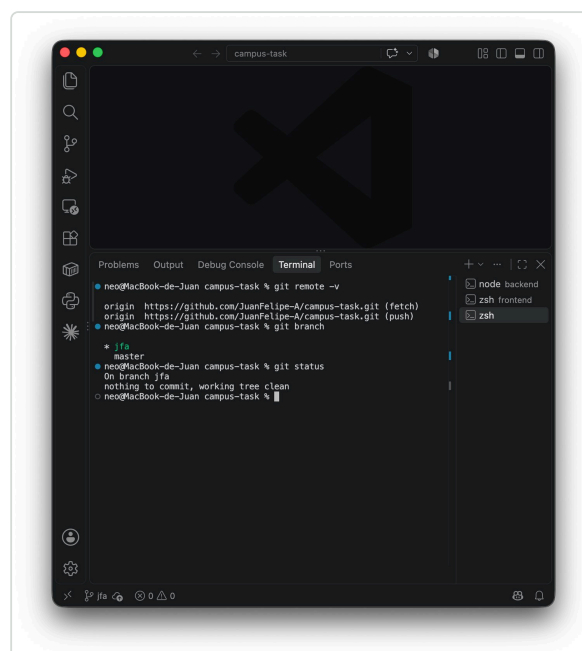
2. Puesta en marcha

2.1 Fork y rama de trabajo

Se hizo fork del repositorio del profesor, se clonó y se creó la rama `jfa`. No se trabajó sobre `master`.



Fork en GitHub



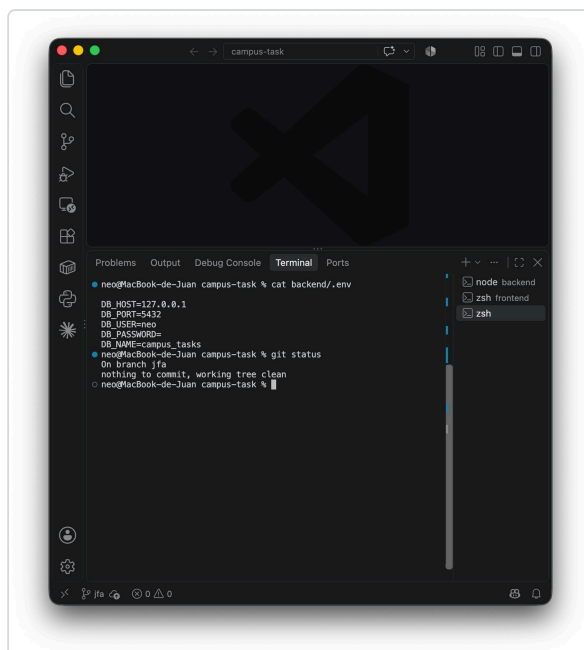
Rama jfa creada

2.2 Variables de entorno

Se creó `backend/.env` (dentro de `backend`, porque `dotenv` lo busca en la carpeta desde donde se arranca el servidor) con la conexión a PostgreSQL local. Estas son las cinco variables que lee `DatabaseService` (solo los nombres; los valores son los de la instalación local):

```
DB_HOST
DB_PORT
DB_USER
DB_PASSWORD
DB_NAME
```

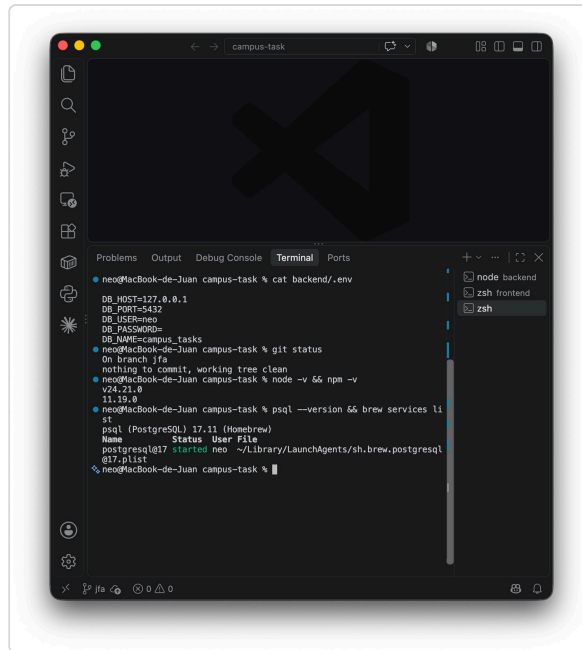
El archivo está ignorado por Git y **no** se sube al repositorio (se verificó con `git status`).



.env ignorado por Git

2.3 Herramientas

- **Node.js 24.x** (el equipo tenía v26; se instaló `nvm` y se fijó Node 24 como versión por defecto). Comprobación: `node -v` → `v24.21.0` y `npm -v` → `11.19.0`.
- **PostgreSQL 17** instalado con Homebrew y levantado como servicio (`brew install postgresql@17` y `brew services start postgresql@17`). Comprobación: `psql --version` → `17.11` y `brew services list` → `postgresql@17 started`.



Versiones de Node, npm y PostgreSQL

2.4 Base de datos

Se entró con `psql` (aparece la consola `postgres=#`) y se ejecutó, una sola vez:

```
CREATE DATABASE campus_tasks;
\c campus_tasks
CREATE TABLE tareas (id SERIAL PRIMARY KEY, titulo TEXT NOT NULL);
INSERT INTO tareas (titulo) VALUES
  ('Leer la guía de la clase 2'),
  ('Preparar el entorno de desarrollo');
```

El `\c campus_tasks` cambia la conexión a la base nueva; así la tabla queda en `campus_tasks` y no en la base equivocada. Comprobación: `SELECT * FROM tareas;` muestra las dos filas.

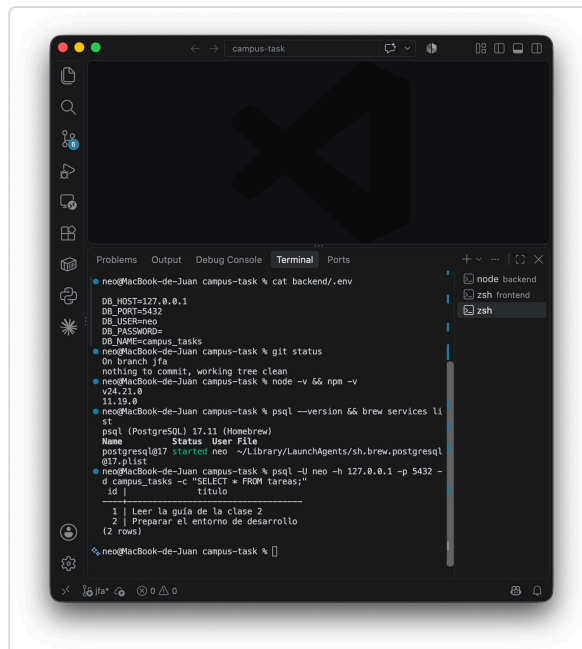
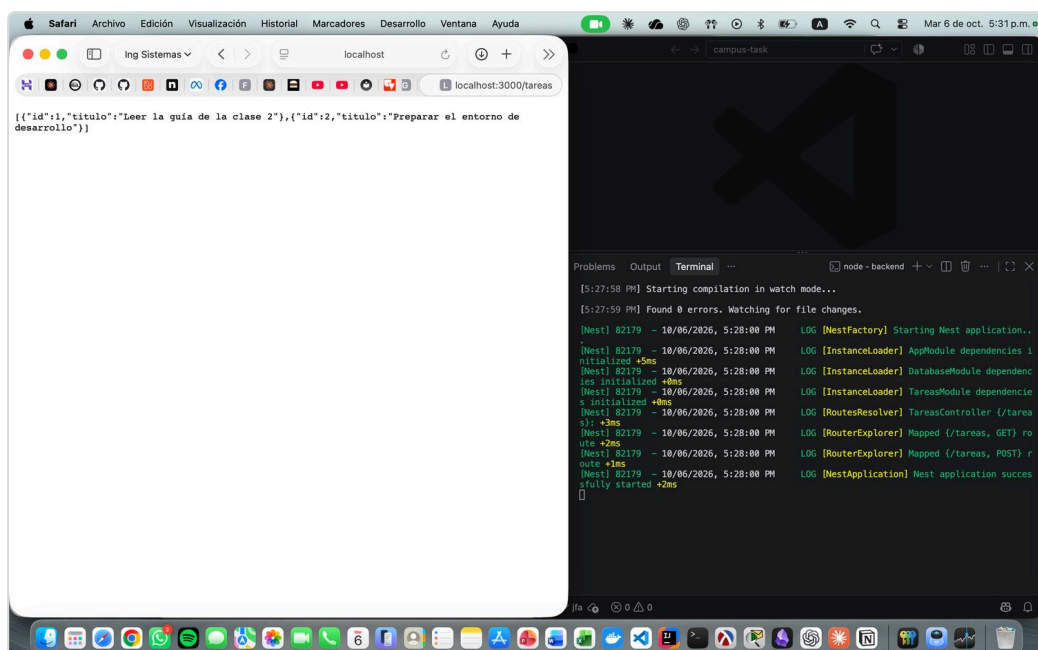


Tabla tareas

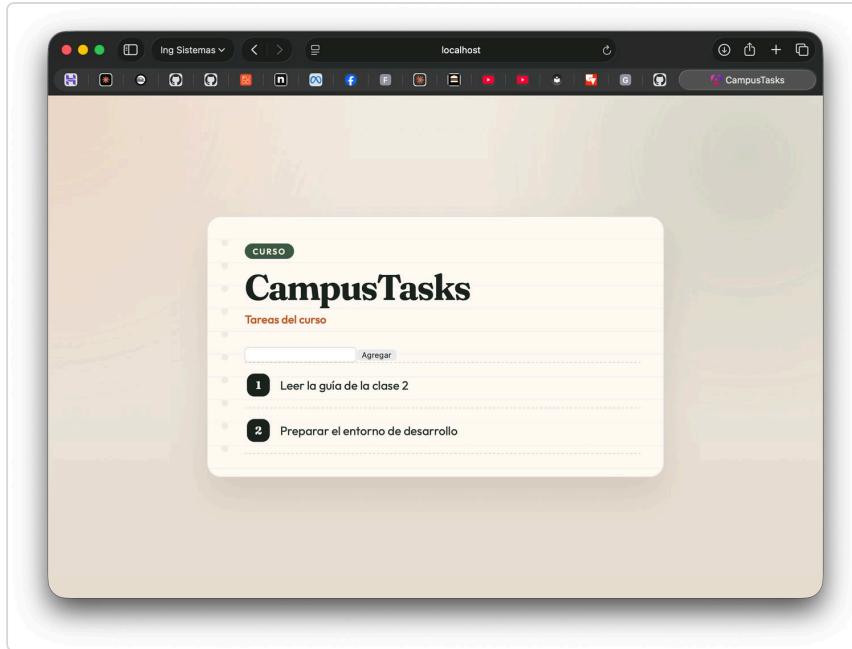
2.5 Backend arrancado

`npm run start:dev` en `backend`. Nest registra las rutas y la API responde en `http://localhost:3000/tareas`.

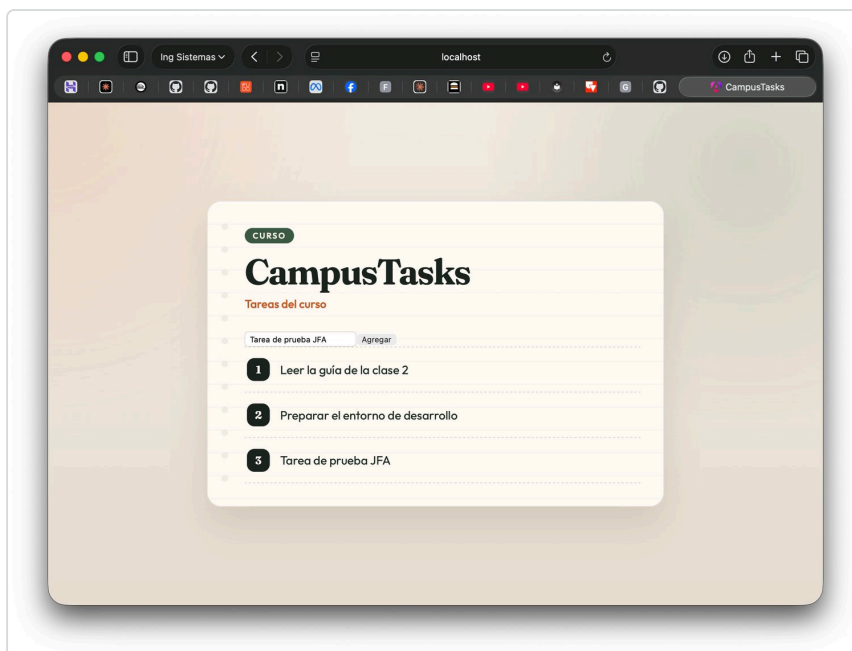


Backend arrancado y respuesta en localhost

2.6 Frontend antes de los cambios



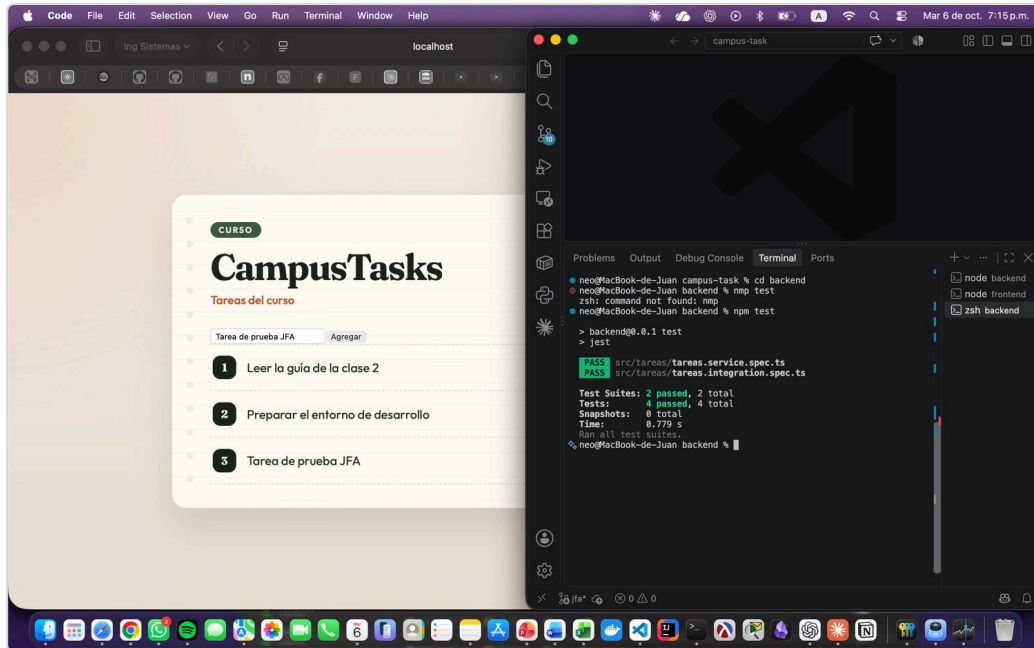
Frontend antes



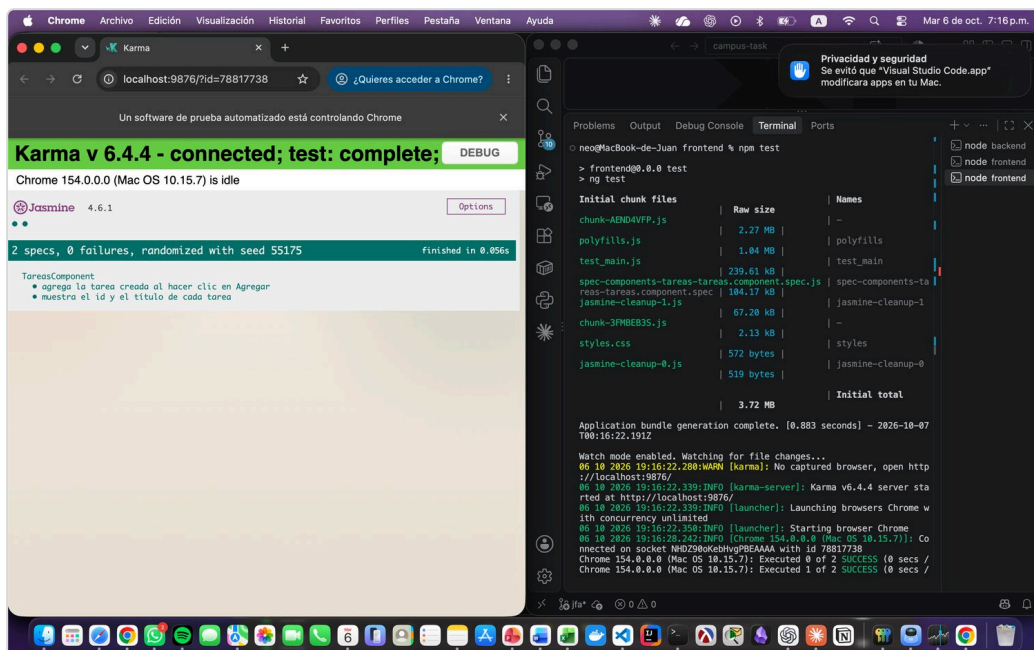
Frontend después de agregar una tarea

2.7 Pruebas iniciales (línea base)

Estado de partida de las pruebas, antes de implementar nada.



npm test backend (inicial)



npm test frontend (inicial)

3. Lectura de los ejemplos

3.1 Backend

Flujo de una petición: `TareasController` → `TareasService` → `DatabaseService.query` (pool de `pg`) → PostgreSQL.

- **Servicio (`tareas.service.ts`)**: cada método ejecuta una consulta SQL **parametrizada** (`$1` , `$2`), nunca concatenada, y devuelve las filas.
- **Controlador (`tareas.controller.ts`)**: recibe la petición HTTP, delega en el servicio y devuelve el resultado. Es la capa que decide los códigos HTTP.
- **Pruebas unitarias (`tareas.service.spec.ts`)**: se reemplaza `query` por un `jest.fn()` que devuelve `{ rows }`; se llama al método del servicio y se verifica el resultado y que `query` se llamó con el SQL y los parámetros exactos. Los ejemplos cubren `listar` (devuelve `resultado.rows` y llama a `query` con el `SELECT` exacto) y `crear` (devuelve la fila insertada y llama a `query` con el `INSERT` y el título como parámetro).
- **Pruebas de integración (`tareas.integration.spec.ts`)**: se levanta la app de Nest con el `DatabaseService` simulado y se usa `supertest` para enviar peticiones HTTP reales contra el controlador; se verifican estado, cuerpo y la llamada a `query`. Los ejemplos cubren `GET /tareas` (200 y el JSON de las filas simuladas) y `POST /tareas` (201, y que `query` recibió el `INSERT` y el título).

Patrón común de todas las pruebas: **preparar** → **ejecutar** → **verificar**.

`backend/test/app.e2e-spec.ts` no se tomó como modelo: es la plantilla de Nest y no sigue el patrón del proyecto.

3.2 Frontend

- **Servicio (`tareas.service.ts`)**: usa `HttpClient` y cada método devuelve un `Observable`. La URL base del backend es `http://localhost:3000`. Hasta el ejemplo, ofrecía `listar` (GET) y `crear` (POST).
- **Componente (`tareas.component.ts`)**: la lista vive en un `signal` (`tareas`). Al crear, la lista solo se actualiza **dentro del `subscribe`**, es decir, cuando el servidor respondió con éxito. Ese es el patrón que se siguió para editar y eliminar.
- **Plantilla (`tareas.component.html`)**: bloque `@for` con el número (`.numero`) y el título (`.titulo`) de cada tarea.
- **Pruebas (`tareas.component.spec.ts`)**: el servicio se reemplaza por un spy de Jasmine (`jasmine.createSpyObj('TareasService', [...])`) cuyos métodos devuelven `of(...)`. Cada prueba prepara el spy, ejecuta una acción sobre el DOM (clic en un botón) y verifica que el spy se llamó con los argumentos esperados y que el DOM refleja el cambio.

3.3 Patrón que se reutilizó

Todas las pruebas nuevas siguen el mismo patrón de los ejemplos, **preparar → ejecutar → verificar**:

Paso	Unitaria (backend)	Integración (backend)	Componente (frontend)
Preparar	<code>query</code> simulado con <code>jest.fn()</code> que devuelve <code>{ rows }</code>	<code>TestingModule</code> con controlador, servicio y <code>DatabaseService</code> simulado	Spy del servicio con lo que debe devolver y componente renderizado con la lista inicial
Ejecutar	Llamar al método del servicio	Petición con <code>supertest</code> (método, ruta y cuerpo)	Escribir en el campo y pulsar el botón
Verificar	Valor devuelto y argumentos exactos de <code>query</code>	Estado, cuerpo y argumentos de <code>query</code>	El spy se llamó con los argumentos esperados y la pantalla muestra el resultado

4. Implementación

4.1 Backend

Servicio (`backend/src/tareas/tareas.service.ts`)

```
async actualizar(id: number, titulo: string): Promise<Tarea> {
  const resultado = await this.db.query<Tarea>(
    'UPDATE tareas SET titulo = $1 WHERE id = $2 RETURNING id, titulo',
    [titulo, id],
  );
  return resultado.rows[0];
}

async eliminar(id: number): Promise<Tarea> {
  const resultado = await this.db.query<Tarea>(
    'DELETE FROM tareas WHERE id = $1 RETURNING id, titulo',
    [id],
  );
  return resultado.rows[0];
}
```

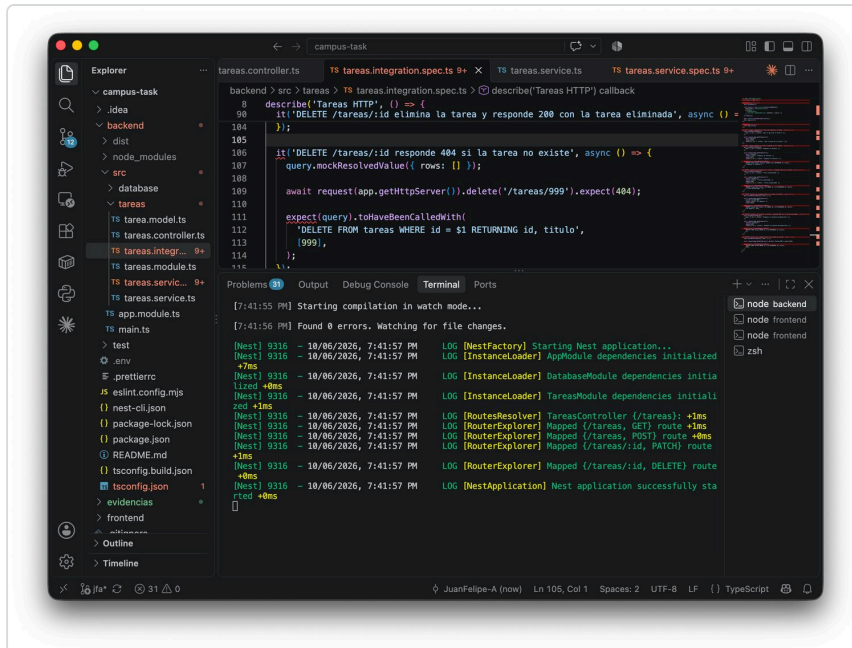
Controlador (backend/src/tareas/tareas.controller.ts)

```
@Patch('/:id')
async actualizar(
  @Param('id', ParseIntPipe) id: number,
  @Body('titulo') titulo: string,
): Promise<Tarea> {
  const tarea = await this.tareasService.actualizar(id, titulo);
  if (!tarea) {
    throw new NotFoundException(`No existe la tarea con id ${id}`);
  }
  return tarea;
}

@Delete('/:id')
async eliminar(@Param('id', ParseIntPipe) id: number): Promise<Tarea> {
  const tarea = await this.tareasService.eliminar(id);
  if (!tarea) {
    throw new NotFoundException(`No existe la tarea con id ${id}`);
  }
  return tarea;
}
```

Rutas expuestas

Método	Ruta	Descripción	Éxito	Error
PATCH	/tareas/:id	Actualiza el título	200 + tarea	404 si no existe, 400 si el id no es numérico
DELETE	/tareas/:id	Elimina la tarea	200 + tarea eliminada	404 si no existe, 400 si el id no es numérico



Rutas registradas por Nest

Decisiones de diseño

1. El `:id` se convierte a número con `parseIntPipe`. Los parámetros de ruta llegan como texto; el pipe los convierte a `number` antes de llegar al servicio, que así recibe el tipo que declara. Además, un id no numérico (`/tarefas/abc`) se rechaza con **400 Bad Request** antes de tocar la base de datos.
2. **Detección del 404 en el controlador, no en el servicio.** Gracias a `RETURNING`, si ninguna fila fue afectada `rows[0]` es `undefined`. El servicio simplemente devuelve ese valor y es el controlador quien decide responder `NotFoundException` (404). Así el servicio no conoce HTTP y se mantiene reutilizable y fácil de probar de forma aislada.
3. **SQL siempre parametrizado** (`$1`, `$2`) para evitar inyección SQL.

4.2 Frontend

Servicio (`frontend/src/components/tareas/tareas.service.ts`)

```
actualizar(id: number, titulo: string): Observable<Tarea> {
  return this.http.patch<Tarea>(`${this.apiUrl}/tarefas/${id}`, { titulo });
}

eliminar(id: number): Observable<Tarea> {
  return this.http.delete<Tarea>(`${this.apiUrl}/tarefas/${id}`);
}
```

```

1 import { Injectable, inject } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4 import { Tarea } from './tarea.model';
5
6 @Injectable({ providedIn: 'root' })
7 export class TareasService {
8   private readonly http = inject(HttpClient);
9   private readonly apiUrl = 'http://localhost:3000';
10
11   listar(): Observable<Tarea[]> {
12     return this.http.get<Tarea[]>(`${this.apiUrl}/tareas`);
13   }
14   crear(titulo: string): Observable<Tarea> {
15     return this.http.post<Tarea>(`${this.apiUrl}/tareas`, { titulo });
16   }
17   actualizar(id: number, titulo: string): Observable<Tarea> {
18     return this.http.patch<Tarea>(`${this.apiUrl}/tareas/${id}`, { titulo });
19   }
20   eliminar(id: number): Observable<Tarea> {
21     return this.http.delete<Tarea>(`${this.apiUrl}/tareas/${id}`);
22   }
23 }
24

```

Código del servicio

Componente (tareas.component.ts y tareas.component.html)

- Tres señales nuevas: `editandoId` (tarea en edición), `tituloEditado` (texto del campo) y `error` (mensaje para el usuario).
- Métodos `editar`, `cancelar`, `guardar` y `eliminar`.
- La plantilla muestra, por cada tarea, **Editar** y **Eliminar**; si la tarea está en edición (`editandoId() === tarea.id`) muestra en su lugar un campo de texto con **Guardar** y **Cancelar**.
- La lista solo cambia dentro del `subscribe exitoso`: `guardar` reemplaza la tarea por la que devolvió el servidor, y `eliminar` la quita de la lista. Si la petición falla, la lista no se toca.

```

1 import { Component, OnInit, inject, signal } from '@angular/core';
2 import { Tarea } from './tarea.model';
3 import { TareasService } from './tareas.service';
4 import { HttpResponse, HttpErrorResponse } from '@angular/common/http';
5
6 @Component({
7   selector: 'app-tareas',
8   standalone: true,
9   templateUrl: './tareas.component.html',
10   styleUrls: ['./tareas.component.css'],
11 })
12 export class TareasComponent implements OnInit {
13   private readonly tareasService = inject(TareasService);
14   tareas = signal<Tarea[]>([]);
15   editandoId = signal<number | null>(null);
16   tituloEditado = signal('');
17   error = signal('');
18
19   ngOnInit(): void {
20     this.cargar();
21   }
22
23   crear(titulo: string) {
24     this.tareasService.crear(titulo).subscribe((tarea) => {
25       this.tareas.update((tareas) => [...tareas, tarea]);
26     });
27   }
28
29   cargar() {
30     this.tareasService.listar().subscribe((tareas) => {
31       this.tareas.set(tareas);
32     });
33   }
34
35   editar(tarea: Tarea) {
36     this.error.set('');
37     this.editandoId.set(tarea.id);
38     this.tituloEditado.set(tarea.titulo);
39   }
40   cancelar() {
41     this.editandoId.set(null);
42   }
43
44   guardar(tarea: Tarea) {
45     this.error.set('');
46     this.tareasService.actualizar(tarea.id, this.tituloEditado()).subscribe({
47       next: (actualizada) => {
48         this.tareas.update((tareas) => {
49

```

Código del componente (1/2)

```

44 guardar(tarea: Tarea) {
45   this.error.set('');
46   this.tareasService.actualizar(tarea.id, this.tituloEditado()).subscribe({
47     next: (actualizada) => {
48       this.tareas.update((tareas) =>
49         tareas.map((t) => (t.id === actualizada.id ? actualizada : t)),
50       );
51       this.editandoId.set(null);
52     },
53     error: (e: HttpResponse) => this.manejarError(e),
54   });
55 }
56
57 eliminar(tarea: Tarea) {
58   this.error.set('');
59   this.tareasService.eliminar(tarea.id).subscribe({
60     next: () => {
61       this.tareas.update((tareas) => tareas.filter((t) => t.id !== tarea.id));
62     },
63     error: (e: HttpResponse) => this.manejarError(e),
64   });
65 }
66
67 private manejarError(e: HttpResponse) {
68   this.editandoId.set(null);
69   if (e.status === 404) {
70     this.error.set('La tarea ya no existe. Se recargó la lista.');
```

Código del componente (2/2)

Comportamiento ante errores (`manejarError`)

- **404** (la tarea ya no existe en el servidor): se muestra "La tarea ya no existe. Se recargó la lista." y se vuelve a pedir la lista, que desaparece de la pantalla lo que ya no existe.
- **Cualquier otro error**: se muestra "No se pudo completar la operación. Intenta de nuevo." y la lista no cambia.
- En ambos casos se sale del modo edición.

4.3 Problemas encontrados durante la integración

Problema	Causa	Solución
Al guardar una edición en el navegador, la petición devolvía 404	El frontend usaba <code>http.put</code> y el backend expone <code>PATCH /tareas/:id</code>	Se cambió a <code>http.patch</code> en <code>TareasService.actualizar</code>
<code>npm test</code> del frontend no compilaba (<code>TS2345</code>)	<code>eliminar</code> estaba tipado como <code>Observable<void></code> , pero el backend devuelve la tarea eliminada y el test simula esa respuesta	Se tipó como <code>Observable<Tarea></code> , según el contrato real del backend
El equipo tenía Node v26 y el proyecto pide v24	Versión instalada por defecto	Se instaló Node 24 con <code>nvm</code>

Los dos primeros solo aparecieron al probar el backend y el frontend **juntos**: cada parte pasaba sus pruebas por separado, lo que muestra el valor de la prueba manual de extremo a extremo.

5. Pruebas

5.1 Pruebas nuevas — backend

Unitarias (servicio) — 2 pruebas nuevas

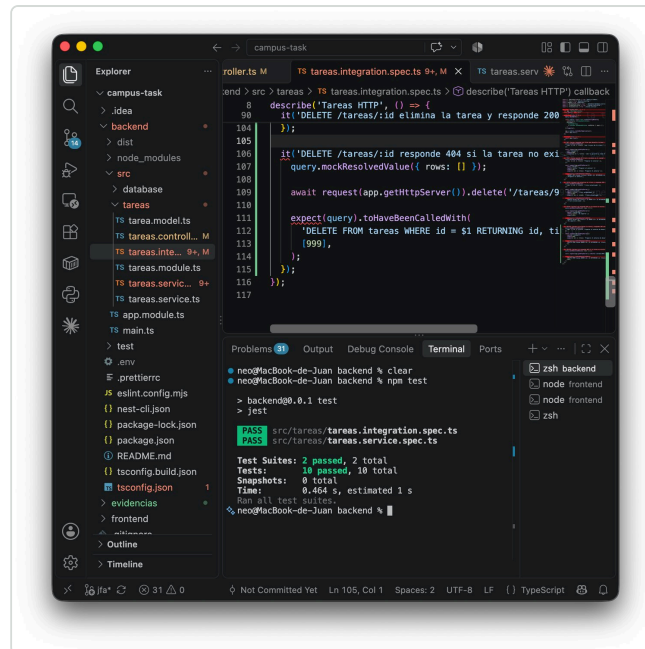
Prueba	SQL verificado	Parámetros
Actualiza el título por id y devuelve la fila actualizada	<code>UPDATE tareas SET titulo = \$1 WHERE id = \$2 RETURNING id, titulo</code>	<code>['Título nuevo', 1]</code>
Elimina la tarea por id y devuelve la fila eliminada	<code>DELETE FROM tareas WHERE id = \$1 RETURNING id, titulo</code>	<code>[2]</code>

Integración (HTTP con supertest) — 4 pruebas nuevas

Prueba	Petición	Resultado esperado
PATCH existente	<code>PATCH /tareas/1 { titulo: 'Título actualizado' }</code>	200, tarea actualizada; query con <code>['Título actualizado', 1]</code>
PATCH inexistente	<code>PATCH /tareas/999</code> , con query devolviendo rows vacío	404; query con <code>['No importa', 999]</code>
DELETE existente	<code>DELETE /tareas/2</code>	200, tarea eliminada; query con <code>[2]</code>
DELETE inexistente	<code>DELETE /tareas/999</code> , con query devolviendo rows vacío	404; query con <code>[999]</code>

En todas se comprueba que el id llega a query como **número**.

Resultado: 2 suites, 10 pruebas pasando (4 originales + 6 nuevas).

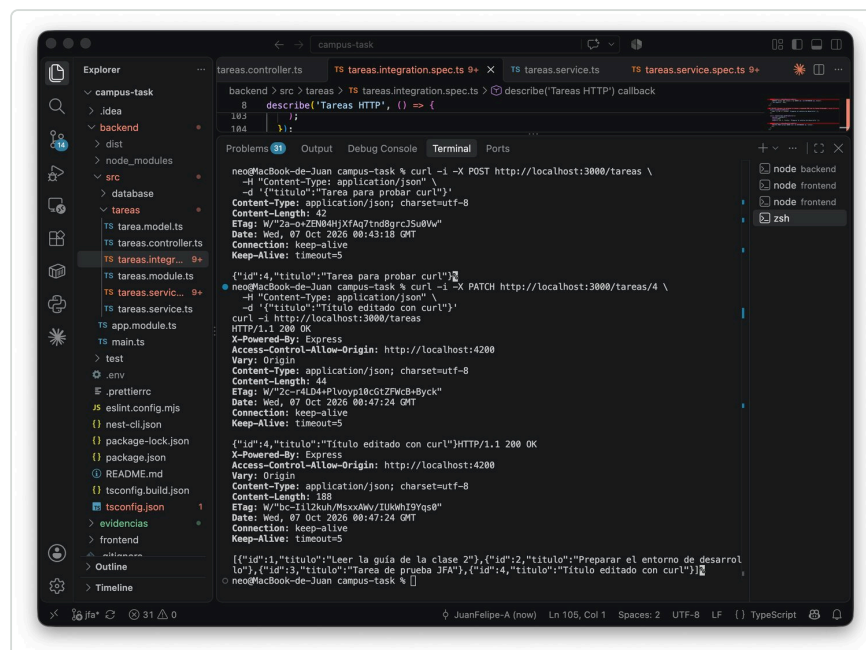


npm test backend final

5.2 Verificación manual de la API con curl

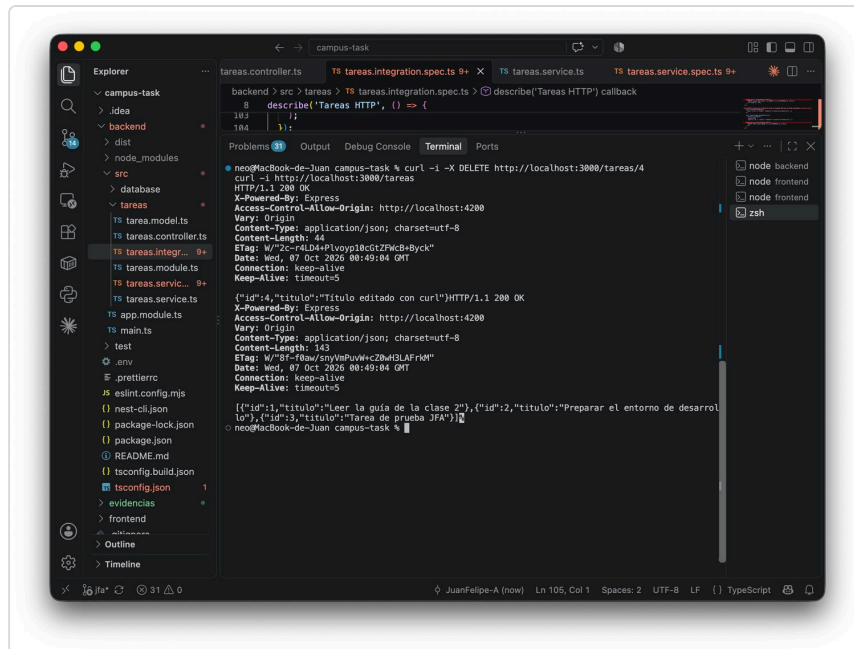
Con el backend y PostgreSQL reales.

PATCH:



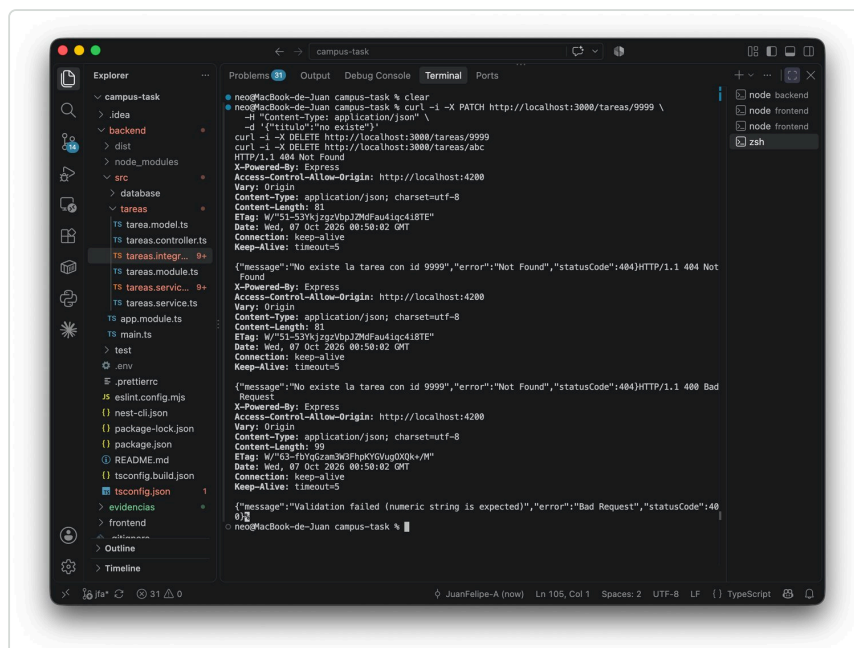
curl PATCH

DELETE:



curl DELETE

Casos de error (id inexistente → 404, id no numérico → 400):



curl errores

5.3 Pruebas nuevas — frontend

El archivo `tareas.component.spec.ts` tiene ahora 4 pruebas (2 originales + 2 nuevas). El spy del servicio incluye `actualizar` y `eliminar`.

Prueba	Qué verifica	Origen
Muestra el id y el título de cada tarea	El DOM pinta <code>.numero</code> y <code>.titulo</code> ; se llamó a <code>listar</code>	original
Agrega la tarea creada al hacer clic en Agregar	<code>crear</code> se llama con el título y la lista muestra las dos tareas	original
Edita una tarea al pulsar Editar y Guardar	Aparece el campo <code>.editar-input</code> ; <code>actualizar</code> se llama con <code>(1, 'Título nuevo')</code> ; la lista muestra el título nuevo	nueva
Elimina una tarea al pulsar Eliminar	<code>eliminar</code> se llama con <code>1</code> ; la lista queda sin tareas	nueva

Resultado: 4 de 4 pruebas exitosas.

```
C:\Users\juanjo\Desktop\campus-task-fe\frontend>npm test -- --watch=false

> frontend@0.0.0 test
> ng test --watch=false

Initial chunk files | Names | Raw size
chunk-AEND4VFP.js | - | 2.27 MB
polyfills.js | polyfills | 1.04 MB
test_main.js | test_main | 239.61 kB
spec-tareas.component.spec.js | spec-tareas.component.spec | 107.82 kB
jasmine-cleanup-1.js | jasmine-cleanup-1 | 67.20 kB
chunk-3FMBEB3S.js | - | 2.13 kB
styles.css | styles | 572 bytes
jasmine-cleanup-0.js | jasmine-cleanup-0 | 519 bytes
Initial total | 3.73 MB

Application bundle generation complete. [2.487 seconds] - 2026-10-07T04:30:39.683Z

06 10 2026 23:30:43.294:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
06 10 2026 23:30:43.296:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
06 10 2026 23:30:43.319:INFO [launcher]: Starting browser Chrome
06 10 2026 23:30:51.525:INFO [Edge 154.0.0.0 (Windows 10)]: Connected on socket zQF-bl-svC3scmQeAAAA with id 90388916
Edge 154.0.0.0 (Windows 10): Executed 4 of 4 SUCCESS (0.655 secs / 0.511 secs)
TOTAL: 4 SUCCESS

C:\Users\juanjo\Desktop\campus-task-fe\frontend>
```

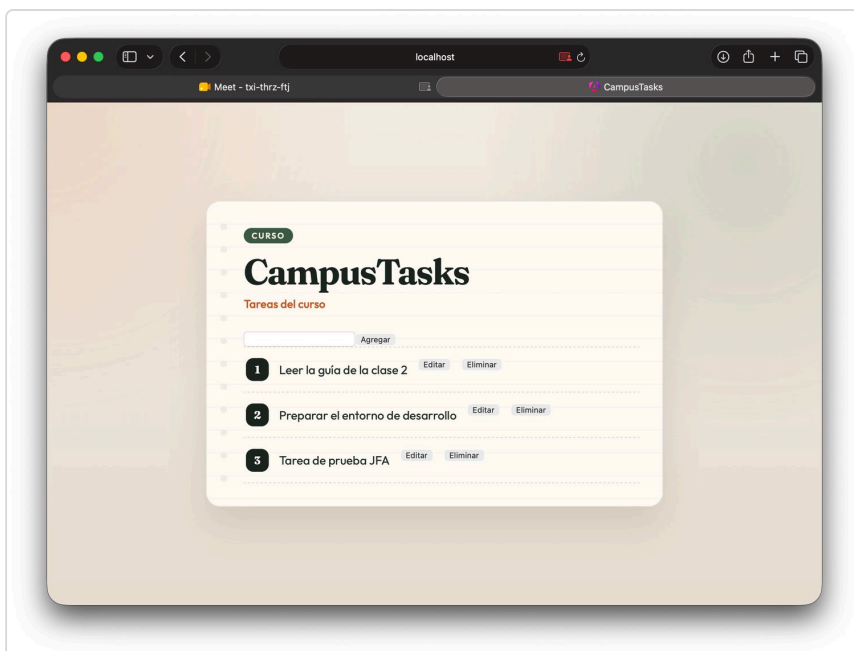
npm test frontend final

5.4 Prueba manual de extremo a extremo

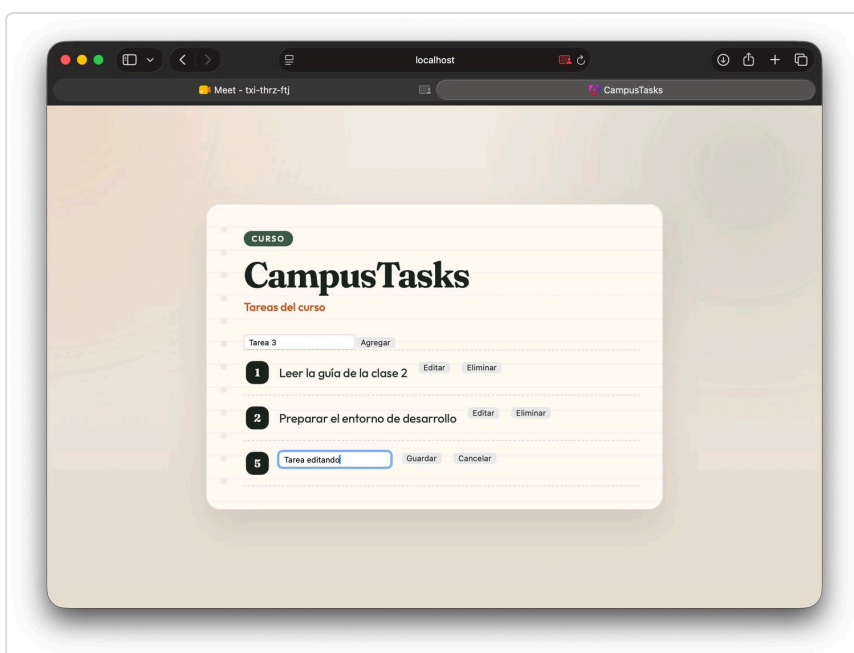
Con PostgreSQL, el backend (`http://localhost:3000`) y el frontend (`http://localhost:4200`) corriendo a la vez.

Edición

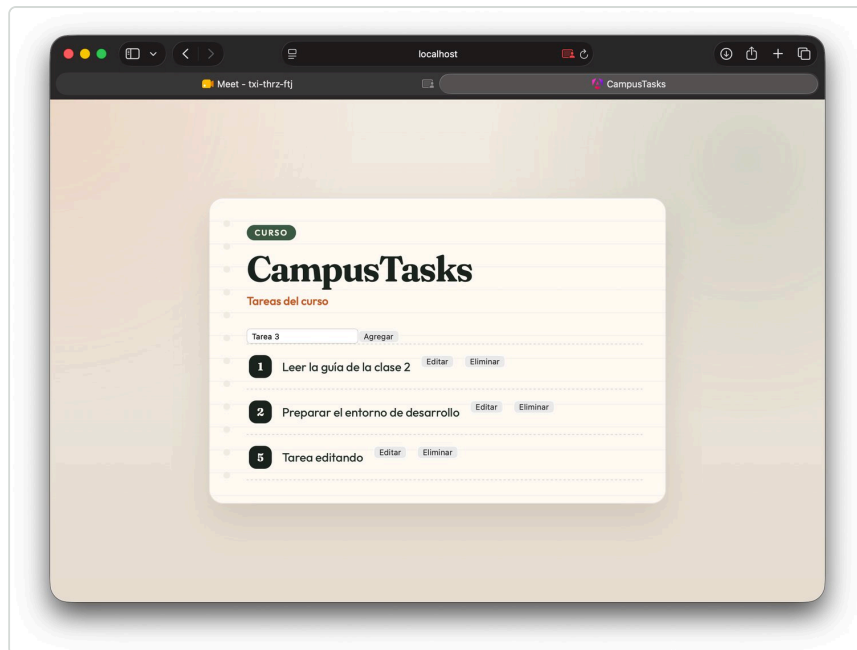
1. Lista inicial en el navegador:



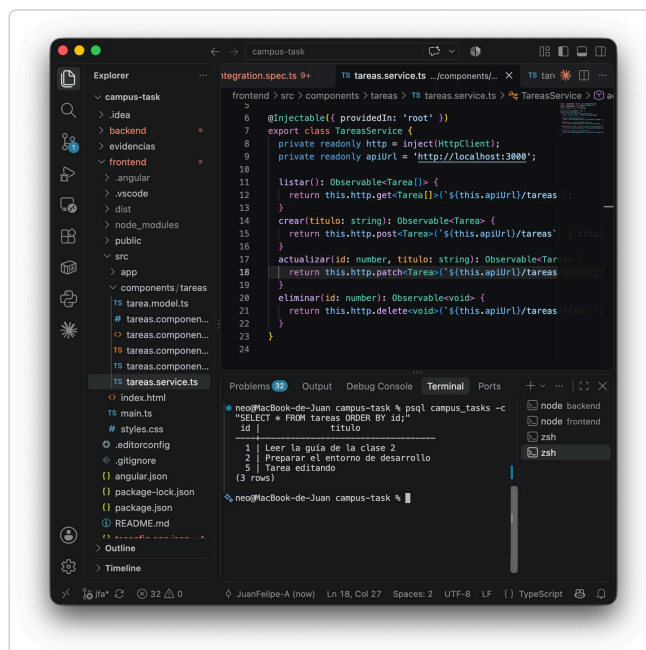
2. Se pulsa **Editar** y se escribe el título nuevo:



3. Tras **Guardar**, la lista muestra el título nuevo sin recargar la página:

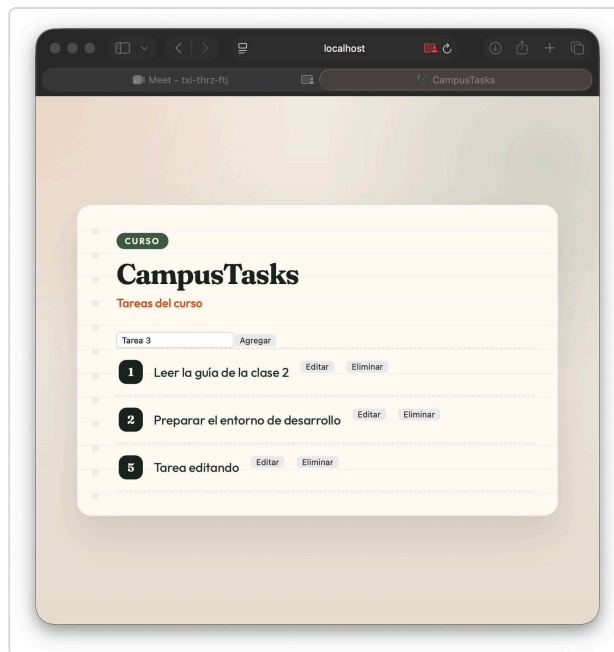


4. La base de datos tiene el cambio:

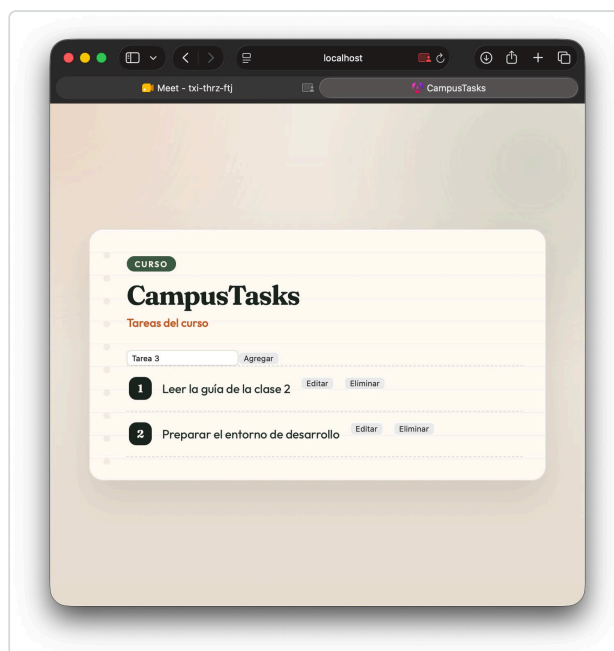


Eliminación

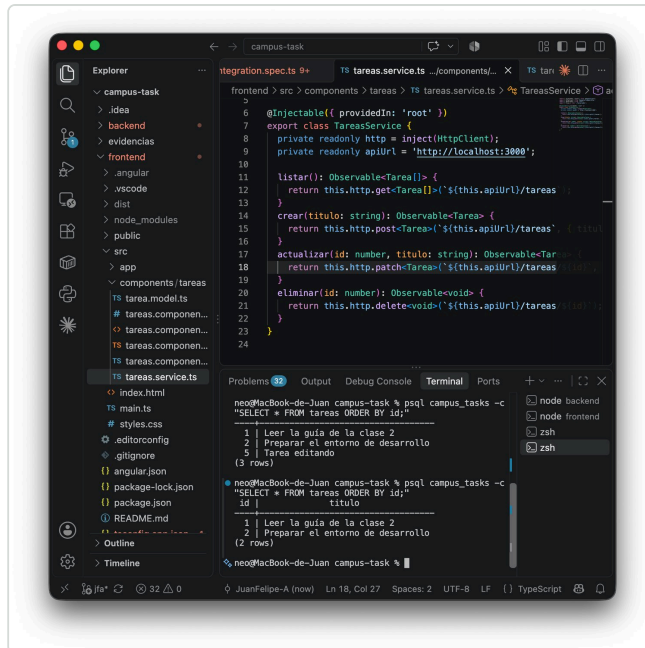
5. Lista justo antes de eliminar:



6. Tras **Eliminar**, la tarea desaparece de la lista sin recargar:



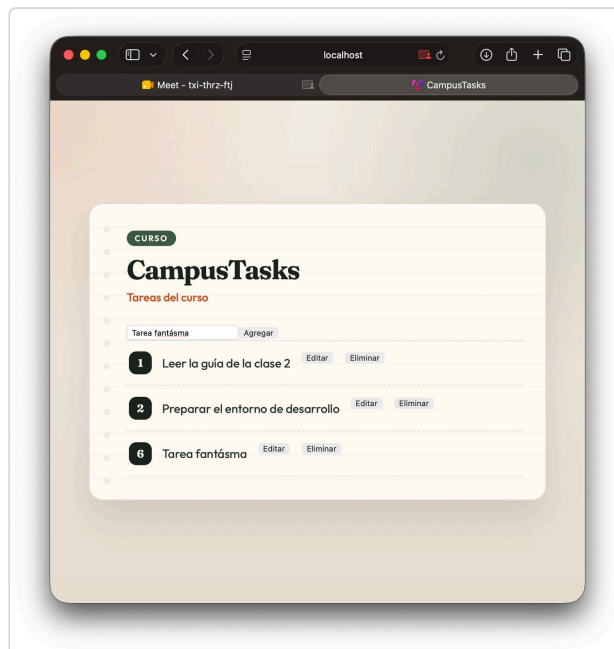
7. La fila ya no está en la base de datos:



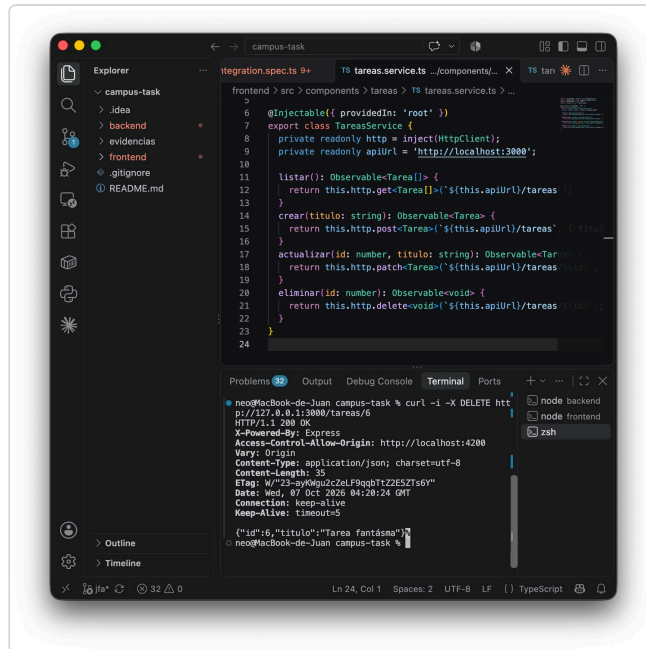
Caso de error: eliminar una tarea que ya no existe (404)

Se crea una tarea desde el frontend, se borra directamente en el backend con `curl` y, **sin recargar la página**, se pulsa **Eliminar** sobre ella.

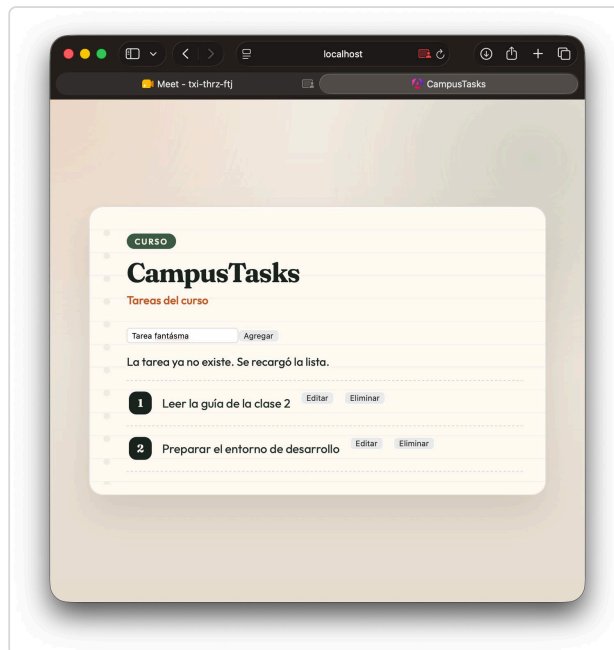
8. Tarea creada y visible en la lista:



9. Se elimina por fuera con `curl` (respuesta 200):



10. Al pulsar **Eliminar**, el backend responde 404; el frontend muestra "La tarea ya no existe. Se recargó la lista." y la lista se actualiza:



6. Git

- Fork: <https://github.com/JuanFelipe-A/campus-task>
- **URL de la rama de entrega:** <https://github.com/JuanFelipe-A/campus-task/tree/jfa>
- Rama de entrega: `jfa`, publicada en el fork. `master` no se modificó.
- `.env` no se versiona.

- Los archivos se agregaron con rutas explícitas desde la raíz del repositorio, y se hizo `git pull --rebase` antes de cada `push`.

Comandos usados

```
# Crear la rama desde master (sin hacer commits en master)
git switch master
git pull
git switch -c jfa

# Antes de cada commit: confirmar que .env no aparece
git status

# Agregar por ruta explícita y hacer commit con mensaje descriptivo
git add backend/src/tareas/tareas.service.ts backend/src/tareas/tareas.service.spec.ts
git commit -m "feat(backend): agrega método actualizar en TareasService con prueba unitaria"

# Integrar los cambios del compañero y publicar
git pull --rebase
git push -u origin jfa      # la primera vez; después basta con git push
```

Commits principales (historial completo: `git log --oneline jfa`)

Área	Commit
Backend	<code>feat(backend):</code> agrega método actualizar en TareasService con prueba unitaria
Backend	<code>feat(backend):</code> agrega ruta PATCH /tareas/:id con 404 y pruebas de integración
Backend	<code>feat(backend):</code> agrega método eliminar en TareasService con prueba unitaria
Backend	<code>feat(backend):</code> agrega ruta DELETE /tareas/:id con 404 y pruebas de integración
Frontend	Agrego actualizar y eliminar en el servicio HTTP
Frontend	Implementacion de edicion de tareas y manejo de errores en TareasComponent
Corrección	<code>fix(frontend):</code> usa PATCH en actualizar para coincidir con el backend
Corrección	<code>fix(frontend):</code> eliminar devuelve Observable<Tarea> según el contrato
Evidencias	<code>docs:</code> agrega pantallazos de evidencia de entorno y backend
Evidencias	<code>docs:</code> agrega pantallazos de la prueba manual de extremo a extremo
Evidencias	<code>docs(frontend):</code> agrega evidencias del servicio, componente y pruebas
Documento	<code>docs:</code> agrega ENTREGA.md y evidencias

```
juanjo@DESKTOP-0411VFM MINGW64 ~/Desktop/campus-task-fe (jfa)
$ git log --oneline -8
git status
5fa4937 (HEAD -> jfa, origin/jfa) docs(frontend): agrega evidencias del servicio, componente y pruebas
72ab494 fix(frontend): eliminar devuelve Observable<Tarea> según el contrato
0e4cf36 docs: agrega pantallazos de la prueba manual de extremo a extremo
261671c test
a93b6a1 fix(frontend): usa PATCH en actualizar para coincidir con el backend
e26325a Implementacion de edicion de tareas y manejo de errores en TareasComponent
a0a5665 Agrego actualizar y eliminar en el servicio HTTP
fafa90b docs: agrega ENTREGA.md y evidencias
On branch jfa
Your branch is up to date with 'origin/jfa'.

nothing to commit, working tree clean
```

git log del frontend